

TP N° 4 : Supervision IoT via MQTT et Node-RED

Objectif :

À la fin de ce TP, vous serez capable de :

- Configurer une carte ESP32 pour mesurer la température et l'humidité via un capteur DHT11
- Publier les données sur un broker MQTT (HiveMQ)
- Souscrire et afficher ces données dans Node-RED à l'aide d'un dashboard graphique
- Commander une LED à distance via MQTT
- Comprendre le principe de communication Publish/Subscribe dans les systèmes IoT

Partie 1 : Configuration du matériel et des outils

1. Matériel nécessaire

Composant	Rôle
ESP32	Microcontrôleur WiFi
DHT11	Capteur température/humidité
LED + résistance 220Ω	Actionneur
Fils Dupont + Breadboard	Connexions

2. Logiciels requis

- **Arduino IDE** (avec carte ESP32 installée)
- **Node-RED** (interface web : <http://localhost:1880>)
- **HiveMQ Web Client** (<https://www.hivemq.com/demos/websocket-client/>)
- **Navigateur web pour visualiser le dashboard** : <http://localhost:1880/ui>

Partie 2 : Configuration de l'environnement Arduino

1. Ouvre **Fichier** → **Préférences**.
2. Dans **URL de gestionnaire de cartes supplémentaires**, ajoute :
http://arduino.esp8266.com/stable/package_esp8266com_index.json
3. Ouvre **Outils** → **Type de carte** → **Gestionnaire de cartes**, cherche **ESP8266 by ESP8266 Community**, installe-la.
4. Sélectionne ta carte :
 - **NodeMCU 1.0 (ESP-12E Module)**
 - ou
 - **Wemos D1 mini**

Étape 2 – Installer les bibliothèques

Dans Croquis → Inclure une bibliothèque → Gérer les bibliothèques :

- DHT sensor library (by Adafruit)
- Adafruit Unified Sensor
- PubSubClient (by Nick O’Leary)

Partie 3 – Architecture MQTT

Rappel du principe Publish/Subscribe :

Élément	Rôle
ESP32 (Publisher)	Envoie les mesures du DHT11 vers le broker
HiveMQ Broker	Transporte les messages entre les clients
Node-RED (Subscriber + Publisher)	Affiche les données et envoie les commandes LED
LED sur ESP32	Réagit aux messages reçus via MQTT

Partie 4 – Indications sur le code Arduino (ESP32)

```
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <DHT.h>

#define DHTPIN 2    // GPIO2 = D4
#define DHTTYPE DHT11

#define LED_PIN 14   // GPIO14 = D5

// ===== WIFI =====
const char* ssid = "Votre_SSID"; // à remplacer par le wifi disponible
const char* password = "Votre_MDP_WIFI";

// ===== MQTT BROKER =====
const char* mqtt_server = "broker.hivemq.com";
const int  mqtt_port = 1883;

WiFiClient espClient;
PubSubClient client(espClient);
DHT dht(DHTPIN, DHTTYPE);

// ===== Topics =====
const char* temp_topic = "iot/fssm/temp"; // ajouter votre nom à fssm
const char* hum_topic = "iot/fssm/hum"; // ajouter votre nom à fssm
const char* led_topic = "iot/fssm/led"; // ajouter votre nom à fssm
```

```
void callback(char* topic, byte* payload, unsigned int length) {
  Serial.print("Message reçu [");
  Serial.print(topic);
  Serial.print("] ");

  String message;
  for (unsigned int i = 0; i < length; i++) {
    message += (char)payload[i];
  }
  Serial.println(message);

  if (String(topic) == "iot/fssm/led") { // ajouter votre nom à fssm
    if (message == "ON") {
      digitalWrite(LED_PIN, HIGH);
      Serial.println("LED allumée");
    } else if (message == "OFF") {
      digitalWrite(LED_PIN, LOW);
      Serial.println("LED éteinte");
    }
  }
}

void reconnect() {
  while (!client.connected()) {
    Serial.print("Connexion MQTT...");
    if (client.connect("ESP8266ClientDHT")) {
      Serial.println("CONNECTÉ");
      client.subscribe(led_topic);
    } else {
      Serial.print("Échec, rc=");
      Serial.print(client.state());
      Serial.println(" > Retry dans 3s");
      delay(3000);
    }
  }
}

void setup() {
  Serial.begin(115200);

  pinMode(LED_PIN, OUTPUT);
  digitalWrite(LED_PIN, LOW);

  dht.begin();
```

```
// --- WIFI ---
WiFi.begin(ssid, password);
Serial.print("Connexion WiFi");
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
  Serial.print(".");
}
Serial.println("\nWiFi connecté");
Serial.print("Adresse IP : ");
Serial.println(WiFi.localIP());

// --- MQTT ---
client.setServer(mqtt_server, mqtt_port);
client.setCallback(callback);
}

void loop() {
  if (!client.connected()) reconnect();
  client.loop();

  float h = dht.readHumidity();
  float t = dht.readTemperature();

  if (!isnan(h) && !isnan(t)) {
    Serial.print("Temp = ");
    Serial.print(t);
    Serial.print("°C Hum = ");
    Serial.println(h);

    client.publish(temp_topic, String(t).c_str());
    client.publish(hum_topic, String(h).c_str());
  }

  delay(3000);
}
```

Partie 5 – Test avec HiveMQ

1. Ouvrir : <https://www.hivemq.com/demos/websocket-client/>
2. Paramètres

Champ	Valeur
Host	mqtt-dashboard.com
Port	8884
SSL	activé
ClientID	webClient_nom

Connection

● connected

Host: mqtt-dashboard.com Port: 8884 ClientID: clientId-7SG5RsnPR Disconnect

Username: Password: Keep Alive: 60 SSL: Clean Session: ☒

Last-Will Topic: Last-Will QoS: 0 Last-Will Retain: ☐

Last-Will Message:

Publish

Topic: aabouhilal/led QoS: 1 Retain: ☒ Publish

Message: ON

Subscriptions

Add New Topic Subscription

Qos: 0

aabouhilal/dht11/temp X

Messages

2025-11-09 17:52:42	Topic: aabouhilal/dht11/temp	Qos: 0
24.10		
2025-11-09 17:52:37	Topic: aabouhilal/dht11/temp	Qos: 0
24.10		
2025-11-09 17:52:32	Topic: aabouhilal/dht11/temp	Qos: 0
24.10		
2025-11-09 17:52:27	Topic: aabouhilal/dht11/temp	Qos: 0
24.10		
2025-11-09 17:52:22	Topic: aabouhilal/dht11/temp	Qos: 0
24.10		
2025-11-09 17:52:17	Topic: aabouhilal/dht11/temp	Qos: 0
24.10		

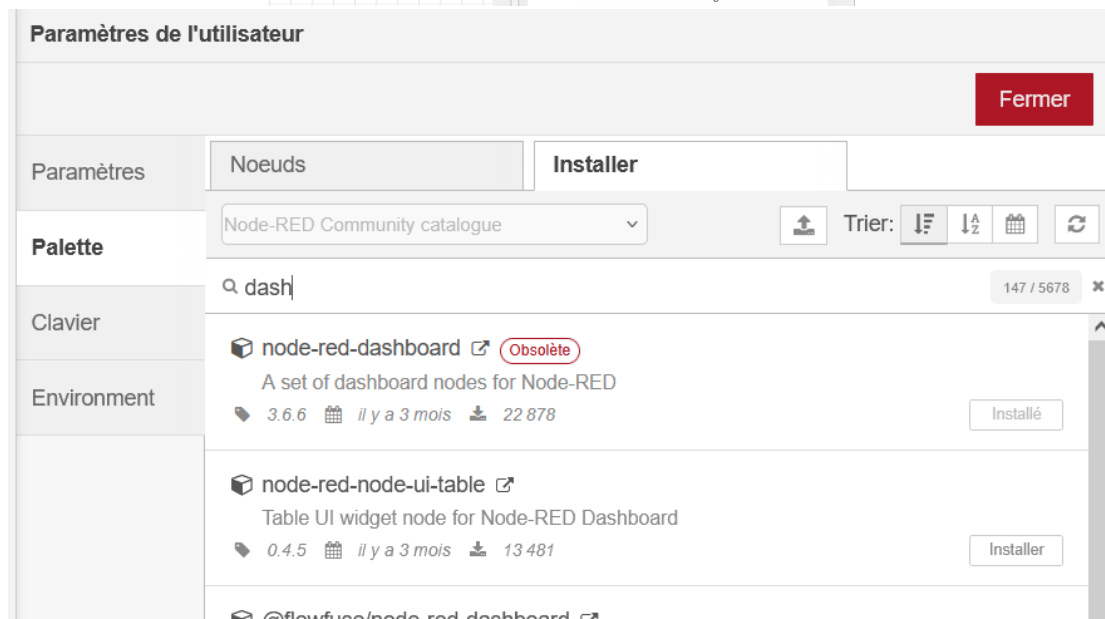
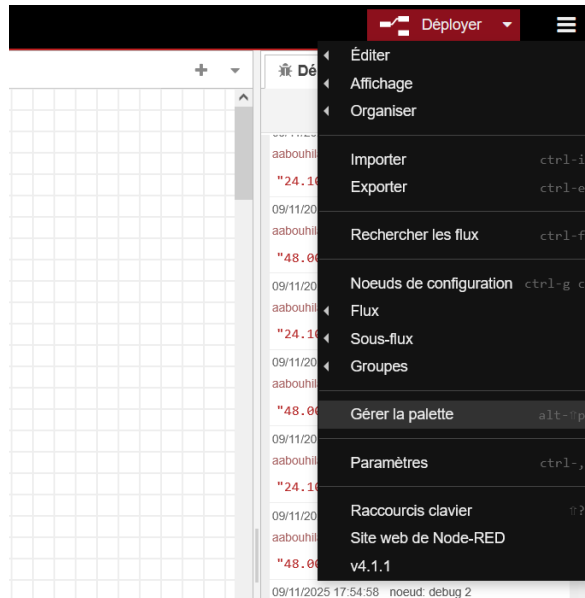
- S'abonner aux topics :
iot/fssmX/temp et iot/fssmX/temp / ajouter votre nom à fssm au lieu de X
- Publier :
 - Topic : iot/fssmX/led / ajouter votre nom à fssm au lieu de X
 - Message : ON / OFF
- Observer le comportement de la LED sur la carte ESP32.

Partie 6 – Supervision sur Node-RED

Étape 1 – Installation du module Dashboard

Dans Node-RED :

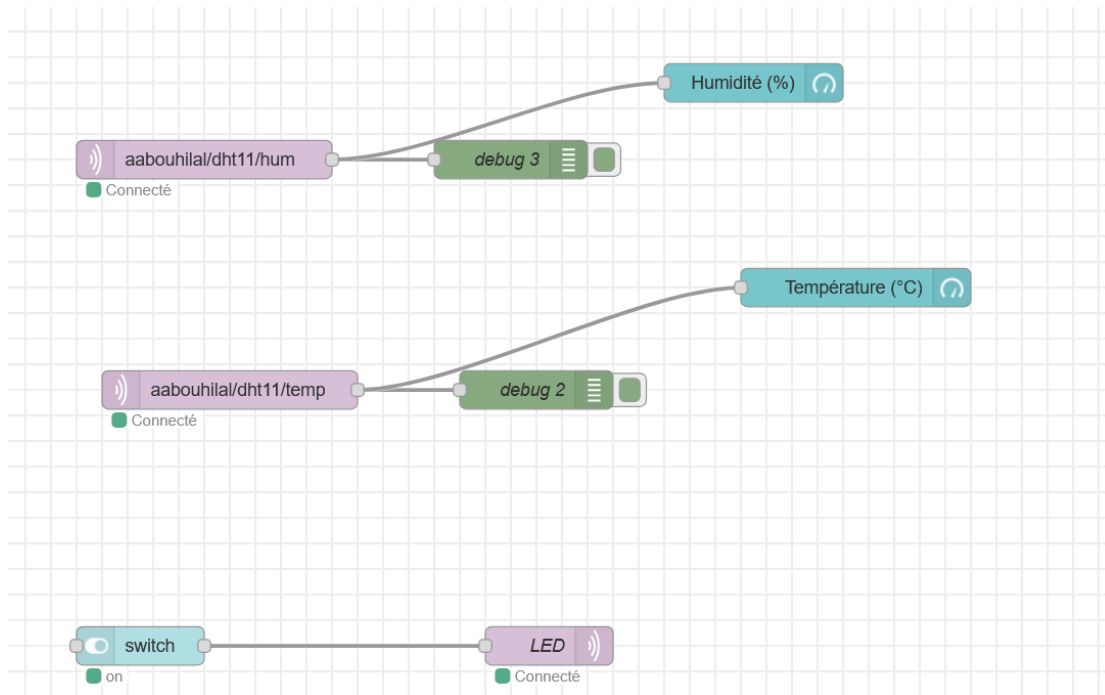
- Menu → *Manage palette* → *Install*
- Chercher et installer : **node-red-dashboard**



Étape 2 – Création du dashboard

1. Nœuds à utiliser :

- mqtt in : reçoit les valeurs depuis HiveMQ
- mqtt out : envoie les commandes LED
- ui_gauge : affiche température
- ui_gauge : affiche humidité
- ui_switch : commande LED



2. Topics à configurer :

Élément	Topic MQTT
Température	etudiantX/dht11/temp
Humidité	etudiantX/dht11/hum
LED	etudiantX/led

3. Créer le dashboard :

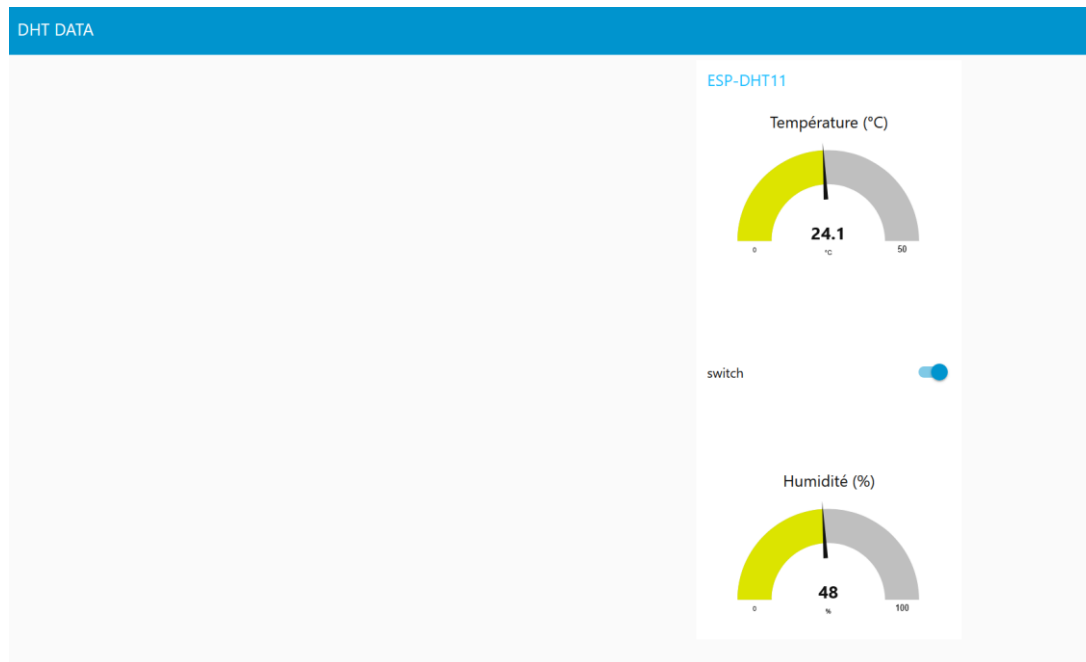
- Onglet (**Tab**) : ESP32 Dashboard
- Groupe (**Group**) : DHT11 Data

4. Déployer le flow.

5. Accéder à : <http://localhost:1880/ui>

Partie 7 – Résultats attendus

- Deux jauges affichent la température (°C) et l'humidité (%).
- L'interrupteur virtuel permet d'allumer ou éteindre la LED de la carte.
- Les valeurs se mettent à jour automatiquement toutes les 5 secondes.



Partie 8 – Analyse et questions

1. Explique le rôle du broker MQTT dans cette architecture.
2. Quelle différence existe entre publish et subscribe ?
3. Pourquoi doit-on donner un ClientID unique à chaque appareil ?
4. Que se passerait-il si plusieurs étudiants utilisent le même topic MQTT ?
5. Propose une extension possible du système (ex. diagnostic de confort ou enregistrement des données).